

0.20.2. Voir x à l'écran : nombre — doublé, deux positions

Les deux, chacun avec son nombre : x en ligne 2, xb en ligne 10.

1. Ce qu'il faut comprendre

C'est le 0.20 et le 0.20.1 réunis : les deux lettres et leurs deux nombres.

Deux variables, deux nombres : `x` s'affiche sous le A (ligne 2), `xb` sous le B (ligne 10).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1704 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;
```

2. UNE VARIABLE — X

```
uint8_t x = 0;           // le A
```

3. UNE VARIABLE — XB

```
uint8_t xb = 0;          // le B
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    x = deplace_x(x, 0, ALPHABET[0], pas);
    nombre(0, 2, x);                // 005 sous le A
    x = deplace_x(x, 0, ALPHABET[0], -pas);
    nombre(0, 2, x);                // 000
    xb = deplace_x(xb, 8, ALPHABET[1], pas);
    nombre(0, 10, xb);              // 005 sous le B
    xb = deplace_x(xb, 8, ALPHABET[1], -pas);
    nombre(0, 10, xb);              // 000

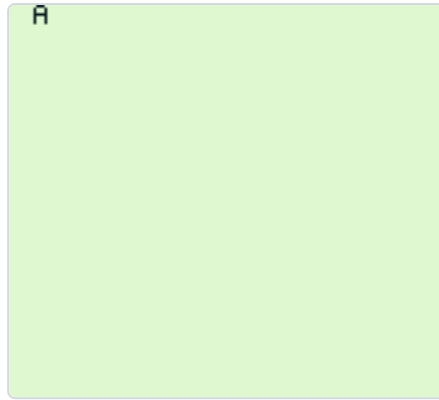
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

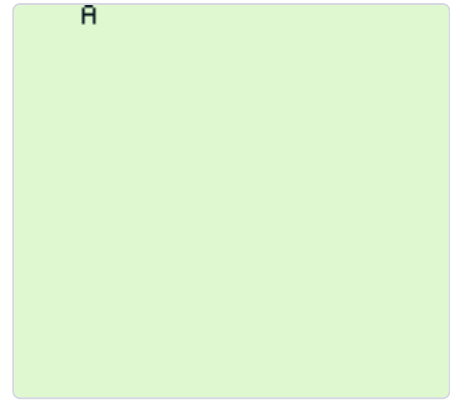
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



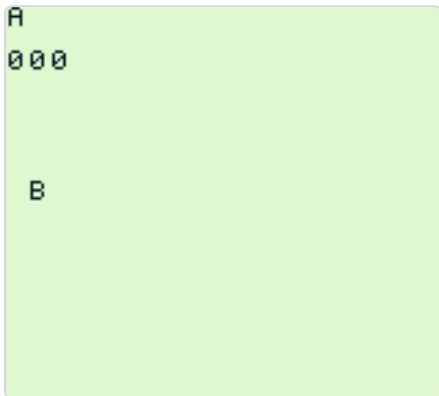
la toute première image — la console vient de démarrer — image 1



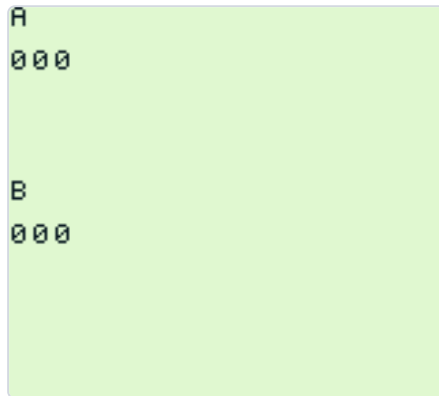
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A puis le B vont et reviennent ; sous chacun, son nombre passe à 005 puis 000.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ 005 sous le A
- ✓ 005 sous le B
- ✓ puis 000 et 000

Ensuite : **0.21. Descendre avec `deplace_y`** — niveau 0 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++